WEATHERHUB

Weather Dashboard and Favourite City Management System

PROJECT REPORT

Submitted by: Tammina Jyothsna Sai

Table of Contents

1. Project Overview 3
2. Objectives 3
3. Technologies Used 3
4. Application Architecture 4
5. Backend Development 4
    5.1 Flask Route Refactoring 4
    5.2 MySQL Database Connectivity 5
    5.3 Create Favourite City API 5
    5.4 Read Favourite Cities API 5
    5.5 Update Favourite City API 5
    5.6 Delete Favourite City API 5
6. REST API Documentation 5
7. Frontend Development 6
    7.1 Dashboard Design 6
    7.2 Current Weather Integration 6
    7.3 Five-Day Forecast Integration 6
    7.4 Favourite City Management 7
    7.5 Responsive Design and User Experience 7
8. Validation, Error Handling and Security 7
9. Testing and Verification 8
10. Screenshots 9
11. Task-wise Questions and Answers 10
    11.1 Backend Tasks 11
    11.2 Frontend Tasks 12
12. Challenges and Learning Outcomes 13
13. Conclusion 13
Appendix A: Additional Screenshots 14

1. Project Overview

WeatherHub is a web application that lets a user search for the current weather of any city, view a five-day forecast, and keep a personal list of favourite cities. I built it as a full-stack project so that I could work on both the server side and the user interface in one application.

The backend is written in Python using Flask and provides a set of REST APIs. MySQL is used to store the favourite cities, so the list is still there after the page is reloaded. The frontend is built with HTML, CSS, Bootstrap 5 and JavaScript. It talks to the backend through the Fetch API, and the backend in turn collects weather data from OpenWeather. This way the browser never contacts OpenWeather directly, and the API key stays on the server.

Apart from the basic weather search, the dashboard also shows a 24-hour temperature trend, air quality, sunrise and sunset details, wind direction, a weather map, and a °C/°F switch. This report documents the work I carried out: how the backend routes were organised, how the database and CRUD operations were set up, how the dashboard was developed, and how everything was tested together.

2. Objectives

The main objectives of this project were:

1. Organise Flask routes into separate modules using Flask Blueprints.
2. Maintain the existing weather API endpoints.
3. Configure MySQL connectivity for storing favourite cities.
4. Implement Create, Read, Update and Delete operations for favourite cities.
5. Validate API requests and handle errors appropriately.
6. Develop a professional, responsive weather dashboard.
7. Integrate current weather and forecast data into the frontend.
8. Test the backend APIs using Postman.
9. Verify the complete frontend and backend integration.
10. Improve usability through responsive layouts, loading indicators and clear notifications.

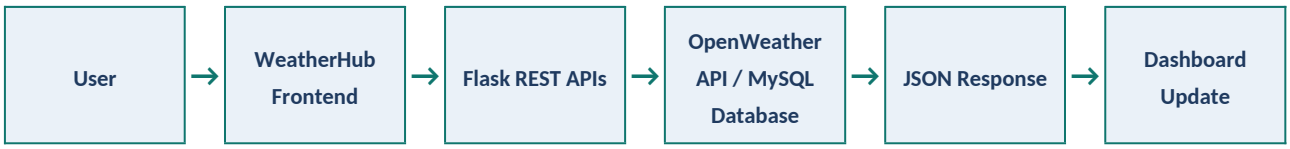
3. Technologies Used

Technology	Purpose
Python	Backend programming
Flask	Backend application and REST API development
Flask Blueprints	Organising routes into separate modules
MySQL	Storing favourite-city information
MySQL Workbench	Creating the table and checking the stored records
HTML5	Frontend structure
CSS3	Styling and responsive design (Grid and Flexbox)
Bootstrap 5 and Bootstrap Icons	Layout, responsive components and icons
JavaScript (ES6+)	Frontend interactions and dynamic page updates
Fetch API	Sending HTTP requests to the Flask endpoints
Leaflet.js	Interactive weather map on the dashboard

Technology	Purpose
OpenWeather API	Retrieving current weather and forecast information
Postman	Testing the REST API endpoints

4. Application Architecture

WeatherHub follows a simple client–server structure. The user interacts with the dashboard in the browser, JavaScript sends requests to the Flask REST APIs, and Flask either fetches data from OpenWeather or reads and writes the MySQL database. The result is returned as JSON, and the dashboard is updated without reloading the page.



Request flow in WeatherHub

The role of each main file and component is given below.

File / Component	Role
`app.py`	Application initialisation, rendering the main page, health check, and registering the Blueprints.
Weather route modules	Separate files for the current weather and five-day forecast endpoints.
`config.py`	Database and application configuration.
`database.py`	Managing the MySQL connection.
`favorites_create.py`	Create operation (POST).
`favorites_read.py`	Read operations (GET all and GET by ID).
`favorites_update.py`	Update operation (PUT).
`favorites_delete.py`	Delete operation (DELETE).
`templates/index.html`	Structure of the dashboard page.
`static/css/style.css`	Styling and responsive layout.
`static/js/app.js`	API calls and dynamic page updates.

5. Backend Development

5.1 Flask Route Refactoring

At the start, the weather routes were handled together with the rest of the application code. I moved the current weather endpoint and the five-day forecast endpoint into their own Python files and registered each one as a Flask Blueprint. The existing endpoint URLs were kept the same, so the weather APIs work exactly as before.

I kept the application initialisation and the health-check endpoint in `app.py`. This file now mainly creates the Flask app, renders the dashboard page and registers the Blueprints, which makes it short and easy to read.

5.2 MySQL Database Connectivity

The favourite cities are stored in a MySQL database named `weatherhub_db`, which contains a table called `favorites`. The structure of the table is shown below, and the stored records can be seen in Figure A1 in the appendix.

Column	Type	Description
`id`	Auto-increment	Unique ID of the favourite (primary key).
`city`	VARCHAR(100), NOT NULL	Name of the city.
`country`	VARCHAR(100), NOT NULL	Country code, for example IN.
`nickname`	VARCHAR(100), NULL	Optional nickname, such as My Home.
`created_at`	TIMESTAMP, default current time	Date and time when the record was created.

To prevent the same place from being saved twice, I added a unique constraint (`uq_favorites_city_country`) on the city and country columns. The API also checks for duplicates before saving so that the user gets a clear message. Database credentials are kept in `config.py` and the connection logic is kept in `database.py`, so the route files only use the database functions and do not deal with connection details.

5.3 Create Favourite City API

`POST /api/favorites` adds a new favourite city. It accepts a JSON request body with the city, the country and an optional nickname. The endpoint first checks that the required fields are present, then confirms that the city and country combination is not already saved. If everything is valid, the record is stored in MySQL and the created record is returned along with a success message. In testing, a valid request returned **201 Created** (see Figure 4).

5.4 Read Favourite Cities API

There are two read endpoints. `GET /api/favorites` returns all saved favourite cities together with a count, and `GET /api/favorites/<id>` returns one specific record. Both return JSON and use parameterised SQL queries, which keeps user input separate from the query text and helps prevent SQL injection. If the requested ID does not exist, the API returns a 404 response with an error message.

5.5 Update Favourite City API

`PUT /api/favorites/<id>` updates an existing favourite. The endpoint validates the new city, country and nickname, checks that the requested record exists, and then updates the matching row. If the updated city and country would match another saved favourite, the request is rejected as a duplicate. A nonexistent ID is reported with an error response, and the updated record is returned when the update succeeds.

5.6 Delete Favourite City API

`DELETE /api/favorites/<id>` removes a favourite city. It deletes only the record that matches the ID given in the URL, so no other rows are affected. After a successful delete a JSON confirmation is returned, and if the ID does not exist a 404 response is returned. Database errors are caught and returned as a clean error message.

6. REST API Documentation

The table below lists the endpoints available in WeatherHub. During development the application runs locally at `http://127.0.0.1:5000`.

HTTP Method	Endpoint	Purpose
GET	`/health`	Check application availability.

HTTP Method	Endpoint	Purpose
GET	`/api/weather/current`	Retrieve current weather information.
GET	`/api/weather/forecast`	Retrieve five-day forecast information.
POST	`/api/favorites`	Create a favourite city.
GET	`/api/favorites`	Retrieve all favourite cities.
GET	`/api/favorites/<id>`	Retrieve a specific favourite city.
PUT	`/api/favorites/<id>`	Update a favourite city.
DELETE	`/api/favorites/<id>`	Delete a favourite city.

Note: replace /health with the exact health-check route used in app.py if it is different, then remove this line.

7. Frontend Development

7.1 Dashboard Design

The dashboard is a single page built with a dark colour palette, and it is divided into the following sections (Figures 1 to 3 and Figure A6):

- A sidebar with WeatherHub branding and navigation links (Home, Current Weather, 24h Trend, Air & Solar, 5-Day Forecast, Favourite Cities and About).
- A header with the city search field, a country selector, the Search button, a recent-searches bar, a light/dark mode switch, a °C/°F switch, and the current date and time.
- A main weather card showing the selected city, temperature, description, feels-like temperature, and the day's high and low.
- Statistics cards for feels-like temperature, humidity, wind speed, pressure and visibility.
- A 24-hour temperature and condition trend chart.
- Cards for sun position and daylight, air quality index, and wind direction.
- The five-day forecast section.
- The Favourite Cities section with Add, Edit and Delete options, and a Weather around the World list.

I tried to keep the design clean and consistent, using the same colours, card style and spacing across all sections. Bootstrap 5 is used for the grid and components, and `style.css` adds the custom styling.

7.2 Current Weather Integration

When the user searches for a city, JavaScript sends a GET request to `/api/weather/current` with the selected city and the country. After the JSON response arrives, the page displays the city and country, temperature, feels-like temperature, humidity, wind speed, pressure, visibility, the weather description and the matching weather icon. While the request is in progress a loading indicator is shown, and if the city is missing or the request fails, a readable error message is displayed instead. The weather logic stays in the backend; the frontend only displays what the API returns.

7.3 Five-Day Forecast Integration

The forecast is requested from `/api/weather/forecast`. The API returns forecast entries at three-hour intervals, so the JavaScript code groups them by date and builds one card per day. Each card shows the day and date, the high and low temperature, the weather description and icon, and the precipitation probability when it is available in the API response (Figure 2). All values come from the actual API data; no forecast values are hardcoded in the page.

7.4 Favourite City Management

The Favourite Cities section loads the saved cities from `/api/favorites` and shows each one with its name, country, optional nickname, and current temperature (Figure 3). From here the user can:

- Add a favourite city using the Add City button and form.
- View the weather of a saved city (keep only if your project loads the weather when a saved city is selected).
- Edit the city details using the edit button.
- Delete a favourite after confirming the action.

After every successful add, edit or delete, the list is fetched again so that the page always matches the database. If there are no saved cities, a short message is shown instead of an empty area, and if a request is unsuccessful the user is told through a notification.

7.5 Responsive Design and User Experience

The layout adjusts for desktop, tablet and mobile screens using Bootstrap's grid and custom media queries, so the cards stack neatly on smaller devices. To improve usability, the dashboard includes loading indicators, consistent notifications for success and error messages, form validation, and a confirmation before deleting a favourite. A light/dark mode switch is provided, and the °C/°F switch changes the units without sending extra requests to the server.

8. Validation, Error Handling and Security

Handling errors properly was an important part of this project. The backend and the frontend deal with the following situations:

- Missing or invalid input fields.
- Invalid JSON request bodies.
- Duplicate favourite cities.
- Invalid IDs and nonexistent records.
- MySQL connection errors.
- Invalid OpenWeather API credentials.
- External service failures and rate limits.
- Network errors and unsuccessful requests on the frontend.

In each case the API returns a JSON error message with a suitable HTTP status code, and the frontend shows that message to the user rather than failing silently. The common status codes are explained below.

Code	Meaning
200	Successful request.
201	Resource created successfully.
400	Invalid request or validation error.
404	Requested resource not found.
409	Duplicate resource or conflicting data, where applicable.
500	Unexpected internal server error.
502 / 503	External service or service availability errors, where appropriate to the implementation.

Not every code is returned by every endpoint. The codes actually observed are recorded during testing in Section 9.

For security, the OpenWeather API key and the database password are kept only on the backend, in the configuration, and are never written in the frontend JavaScript. They should also not be uploaded in public source code. Since all weather requests go through Flask, the browser never sees these values.

9. Testing and Verification

I created a Postman collection named "WeatherHub REST API Testing" to check the API requests, the JSON responses and the HTTP status codes of every endpoint. For example, a valid POST request to `/api/favorites` returned **201 Created** with the new record (Figure 4), and a GET request returned **200 OK** with all saved favourites (Figure A2). The saved records were also checked directly in MySQL Workbench (Figure A1). Browser testing was then used to verify the complete user workflow, from searching a city to adding, editing and deleting favourites. The Result column is left for the final entry after each test is carried out.

Test Area	Test Cases	Result
1. Health check	Successful response	☐ Pass ☐ Fail
2. Current weather	Valid city, missing city, invalid city	☐ Pass ☐ Fail
3. Five-day forecast	Valid city, invalid city	☐ Pass ☐ Fail
4. Create favourite	Valid JSON, missing fields, duplicate city	☐ Pass ☐ Fail
5. Read favourites	Retrieve all records, empty database	☐ Pass ☐ Fail
6. Read one favourite	Existing ID, nonexistent ID	☐ Pass ☐ Fail
7. Update favourite	Valid update, invalid data, nonexistent ID	☐ Pass ☐ Fail
8. Delete favourite	Existing ID, nonexistent ID	☐ Pass ☐ Fail
9. Frontend integration	Search, forecast, add, edit, delete and reload	☐ Pass ☐ Fail
10. Responsive layout	Desktop and mobile	☐ Pass ☐ Fail
11. Browser console	Check for unhandled JavaScript errors	☐ Pass ☐ Fail

10. Screenshots



Figure 1: WeatherHub Dashboard

The main dashboard showing the current weather card and the weather statistics.

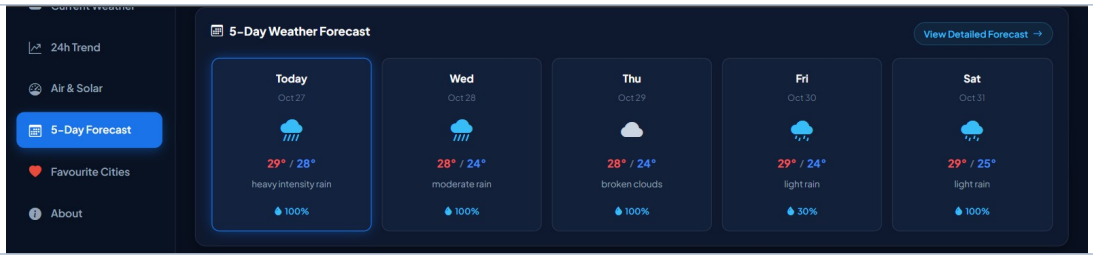


Figure 2: Five-Day Forecast

Forecast cards showing the day, high and low temperature, description, icon and rain probability.

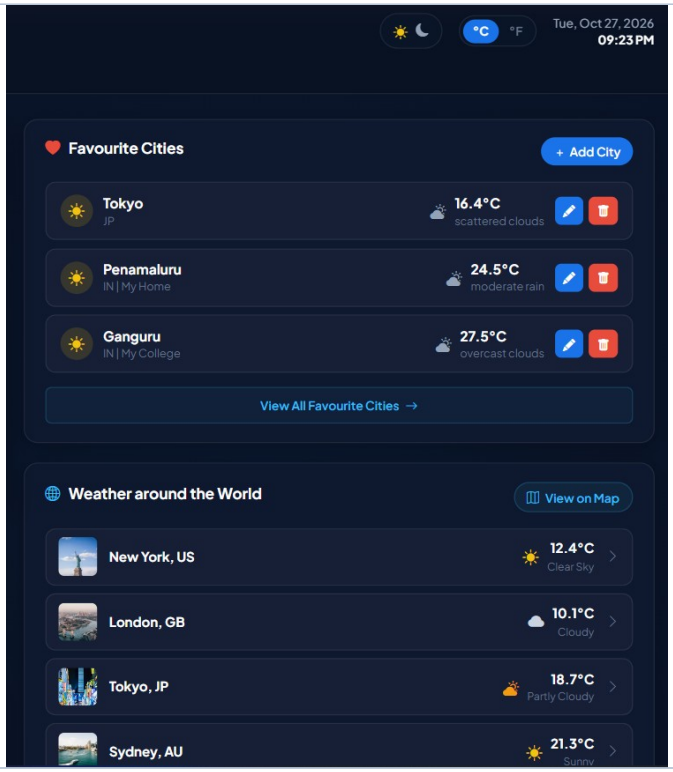


Figure 3: Favourite City Management

Saved favourite cities with the Add City, Edit and Delete actions.

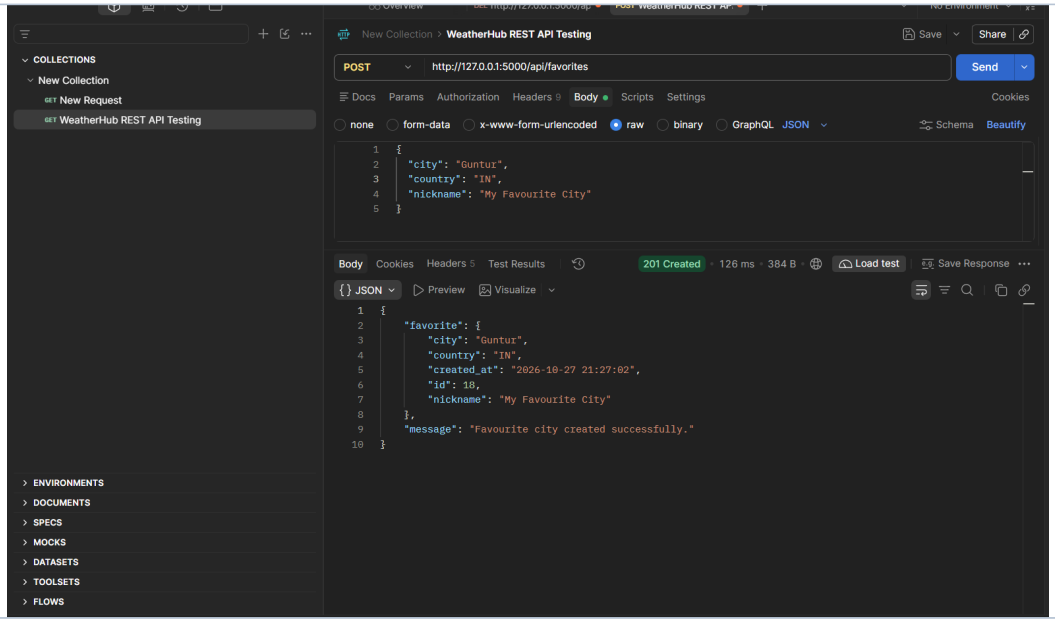


Figure 4: REST API Testing Using Postman

A POST request to /api/favorites returning 201 Created with the new record in JSON.

11. Task-wise Questions and Answers

This section answers the question given in each task of the backend and frontend assignments. Backend Tasks 2 and 8 do not have a question; they are covered in Sections 5.2 and 9.

11.1 Backend Tasks

Task 1 – *How can you organize Flask routes into separate files while keeping the same endpoint URLs?*

I created a separate Python file for each endpoint and defined a Blueprint in it. The route functions were moved into these files with the same URL paths in the route decorators, for example `/api/weather/current` and `/api/weather/forecast`. In `app.py` I imported the Blueprints and registered them using `app.register_blueprint()`.

Because the paths did not change, the frontend and Postman requests continue to work without any change. Application initialisation and the health-check endpoint stayed in `app.py`.

Task 3 – *How can you receive JSON data from a POST request and save it in MySQL using Flask?*

In the route I read the request body using `request.get_json()`. I checked that city and country were present and not empty, and that the body was valid JSON. Then I opened a database connection through `database.py` and ran a parameterised `INSERT` query with the values passed separately from the SQL text.

After committing the change, the new row is read back and returned using `jsonify()` with status 201. If a required field is missing the API returns a 400 response, and a duplicate city and country combination is rejected with a conflict error.

Task 4 – *How can you retrieve all records or a specific record from MySQL and return them as JSON?*

For all records, the route runs a `SELECT` query on the favorites table, fetches all rows and returns them with `jsonify()`. My response contains a count and a list of favourites (Figure A2), and an empty table gives an empty list instead of an error.

For one record, the ID from the URL is passed to a parameterised query with `WHERE id = %s`. If no row is found, the API returns a 404 response; otherwise the record is returned as JSON.

Task 5 – *How can you update a MySQL record using a PUT request and identify whether the requested record exists?*

The route first validates the JSON body and then runs a `SELECT` for the given ID. If no row comes back, the record does not exist and a 404 response is returned. Next I check that the new city and country do not already belong to a different record, and then run an `UPDATE ... WHERE id = %s` query.

After the update is committed, the row is selected again and the updated record is returned. Checking existence with a `SELECT` is more reliable than relying only on the number of affected rows, because MySQL can report zero changes when the new values are the same as the old ones.

Task 6 – *How can you delete a particular MySQL record using a DELETE request without affecting other records?*

The ID is taken from the URL and used in a parameterised query, `DELETE FROM favorites WHERE id = %s`. Since the condition uses the primary key, only that one row can be deleted and all other records stay unchanged.

If the query affects no rows, the record does not exist and a 404 response is returned. On success a JSON confirmation message is sent, and any database error is caught and returned as a clean error response.

Task 7 – *How should a REST API communicate success, client errors, missing resources, and external service failures?*

A REST API should use both the HTTP status code and a clear JSON message. Success is shown with 200 (OK) or 201 (Created). Client mistakes, such as missing fields or invalid JSON, use 400, and duplicate data uses 409. A missing resource uses 404.

Failures that are not the client's fault use 500 for unexpected server errors and 502 or 503 when an external service such as OpenWeather fails or is unavailable. Messages must never include the API key, database password or internal details, so that no secret information is exposed.

11.2 Frontend Tasks

Task 1 – *How can you organize HTML, CSS, and JavaScript files in a Flask application?*

Flask looks for HTML pages in the `templates` folder and for static files in the `static` folder. So I kept `index.html` in `templates`, `style.css` in `static/css` and `app.js` in `static/js`.

The route for `/` uses `render_template('index.html')`, and the CSS and JavaScript are linked using `url_for('static', filename=...)` so that the paths stay correct. The existing API routes were not changed.

Task 2 – *How can you design a professional, responsive weather dashboard using HTML, CSS, and Bootstrap 5?*

I used Bootstrap's grid system and utility classes for the layout, and Bootstrap Icons for the icons. The sidebar, header, weather card, statistics cards, forecast and favourites are all placed in cards with the same spacing and rounded corners.

Colours are kept consistent using a dark palette defined in `style.css`, and media queries adjust the layout so that the cards stack on tablet and mobile screens.

Task 3 – *How can JavaScript Fetch API retrieve weather data from a Flask REST endpoint and display it dynamically?*

When the search form is submitted, JavaScript stops the default page reload, reads the city and country, and calls `fetch('/api/weather/current?city=...&country=...')`. The response is converted using `response.json()` and checked with `response.ok`.

The values are then placed into the page elements using JavaScript, so the page updates without a reload. A loading indicator is shown before the request and hidden afterwards, and an empty city name or an API error shows an error message.

Task 4 – *How can you display forecast data returned by a REST API in dynamically generated HTML cards?*

After fetching `/api/weather/forecast`, I loop through the three-hour entries and group them by their date. For each date I find the highest and lowest temperature, the main description and icon, and the precipitation probability if the API provides it.

One card is created for each day using template strings and inserted into the forecast container. If the data is missing or the request fails, a message is shown instead of cards.

Task 5 – *How can you retrieve favourite cities from a Flask REST API and display them dynamically on a dashboard?*

A function calls `GET /api/favorites` and loops through the returned list. For each favourite it creates a row showing the city, country, nickname and weather, along with Edit and Delete buttons that carry the record ID.

If the list is empty, a friendly message is displayed. The same function is called again after any add, edit or delete so the list is always refreshed.

Task 6 – *How can you submit JSON data from a frontend form to a Flask REST API using the POST method?*

On form submit I read the input values, trim them and check that the city and country are not empty. Then I call `fetch('/api/favorites', { method: 'POST', headers: { 'Content-Type': 'application/json' }, body: JSON.stringify(data) })`.

The response decides the message shown to the user: a success notification and a refreshed list for a saved city, or the error message from the API for duplicates and invalid data.

Task 7 – *How can you update a saved favourite city using a PUT request from JavaScript?*

When the Edit button is clicked, the existing city, country and nickname are filled into the form. On submit, JavaScript sends a `fetch` request with the method PUT to `/api/favorites/<id>` and the new values as JSON.

A successful response closes the form and refreshes the list, while validation errors or duplicates are shown as notifications.

Task 8 – *How can you delete a favourite city from the frontend by calling a Flask REST API with the DELETE method?*

The Delete button first asks the user to confirm. If the user agrees, JavaScript sends `fetch('/api/favorites/' + id, { method: 'DELETE' })`.

A success or failure message is displayed and the list is reloaded. If the server returns 404 because the record was already removed, the user is informed and the list is refreshed anyway.

Task 9 – *How can you make a REST API-driven dashboard user-friendly, responsive, and accessible?*

I added loading indicators and disabled buttons while a request is running so that users cannot send the same request twice. Success and error messages appear in one consistent notification area, and network errors or invalid JSON responses are caught using `try...catch`.

The dashboard has a light/dark mode switch, a responsive layout for all screen sizes, and a °C/°F switch that converts the values without calling the API again. Using proper form elements, labels and buttons helps with keyboard use, and the OpenWeather API key stays only on the backend.

Task 10 – *How can you verify that all frontend operations work correctly with the existing Flask REST APIs?*

I tested the complete workflow in the browser: searching different cities and checking that the weather and forecast match the selected city, then adding, editing and deleting a favourite and confirming that each change appears on the page and stays after a reload.

I also tried invalid city names and empty form submissions, and checked the browser developer tools for unhandled JavaScript errors on desktop and mobile layouts. The results are recorded in the testing table in Section 9.

12. Challenges and Learning Outcomes

Working on WeatherHub gave me hands-on experience with how a complete web application is put together. Through this project I learned:

- How to organise a Flask application into separate modules using Blueprints.
- How to connect a Python application to a MySQL database.
- How to develop REST APIs and implement CRUD operations.
- How to send JSON requests and handle responses.
- How to use the JavaScript Fetch API to update a webpage dynamically.
- How to integrate external weather data into an application.
- How to handle invalid input, duplicate records and missing resources.
- How to test APIs using Postman.
- How to build responsive interfaces and connect frontend operations with database changes.

The most useful lesson for me was that the frontend, backend and database all need to work together, so a small mistake in one place, such as a wrong status code or an unhandled error, can affect the whole user experience. Testing each API separately in Postman before connecting it to the page made this much easier to manage.

13. Conclusion

WeatherHub combines current weather information, five-day forecasts and favourite-city management in one web application. It uses Flask REST APIs for the backend, MySQL for storing data, and JavaScript with a responsive frontend design to give the user a simple and clear dashboard.

Completing this project gave me practical experience in backend development, database operations, API integration, testing and user-interface development. It also helped me understand how these parts work together in a real application.

Appendix A: Additional Screenshots

These screenshots support the database, API testing and extra dashboard features described in the report.

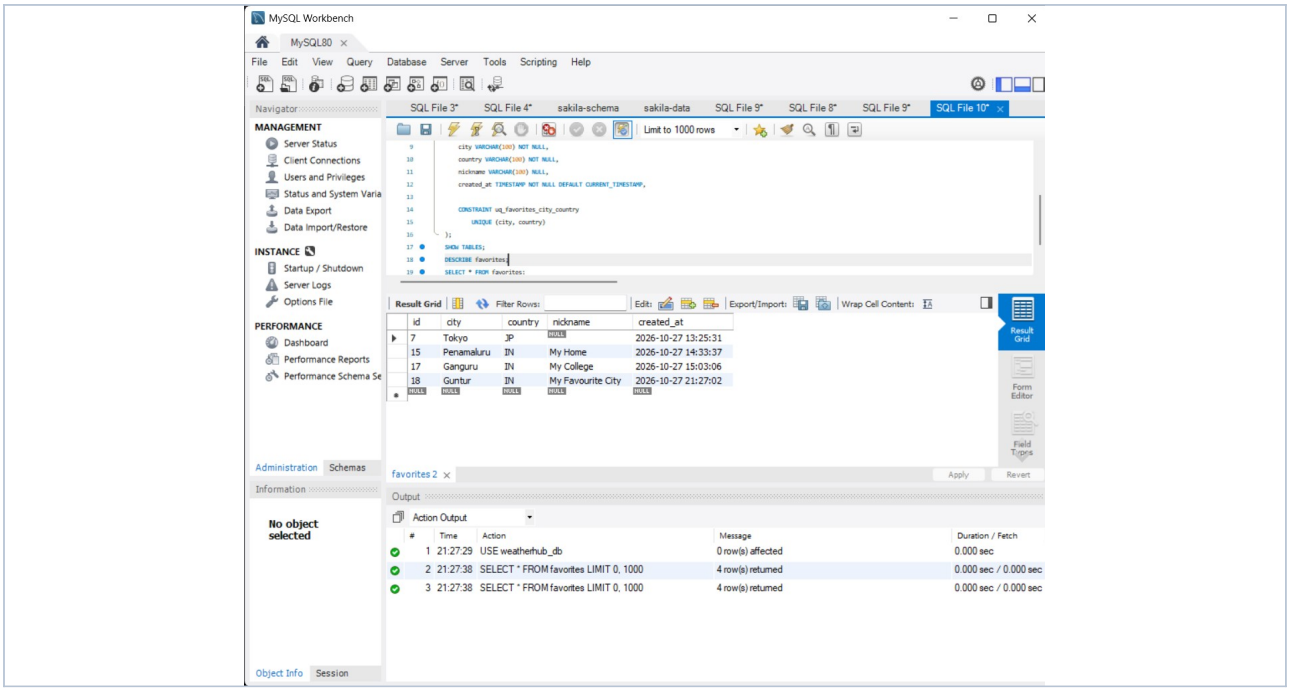


Figure A1: MySQL Workbench – favorites Table

The favorites table in weatherhub_db showing the saved records and the table definition.

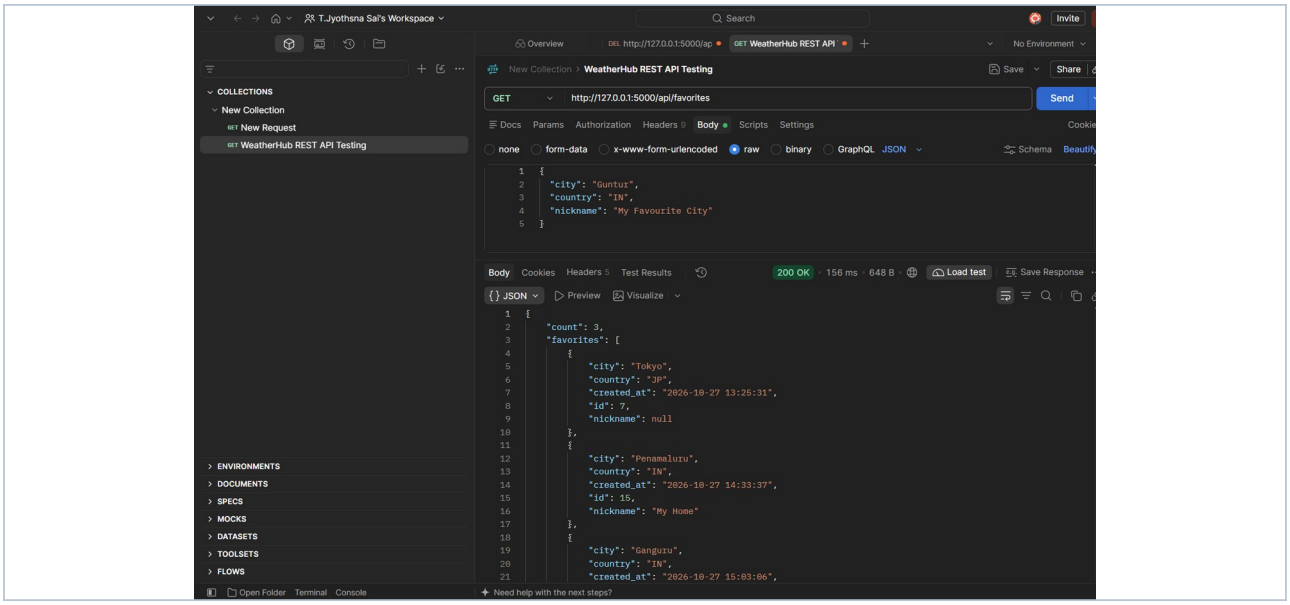


Figure A2: Postman – Retrieve All Favourites

A GET request to /api/favorites returning 200 OK with the count and the list of favourites.



Figure A3: 24-Hour Temperature Trend

Hourly temperature chart with condition icons for the selected city.



Figure A4: Sun Position, Air Quality and Wind Direction

Additional dashboard cards for solar details, air quality index and wind compass.

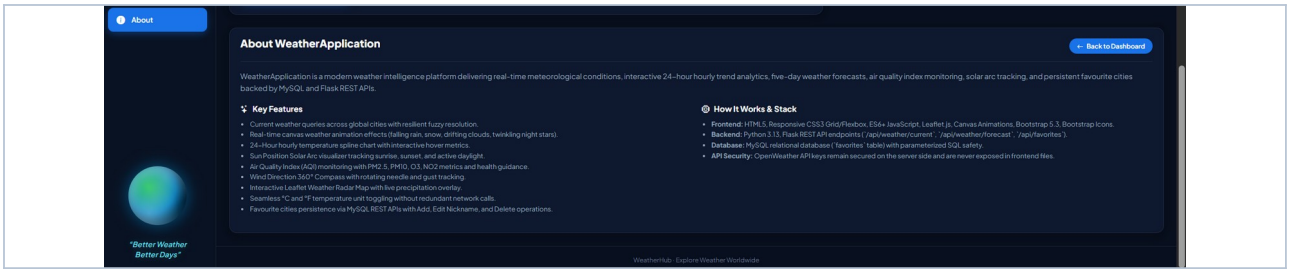


Figure A5: About Page

The About section listing the features and the technology stack.

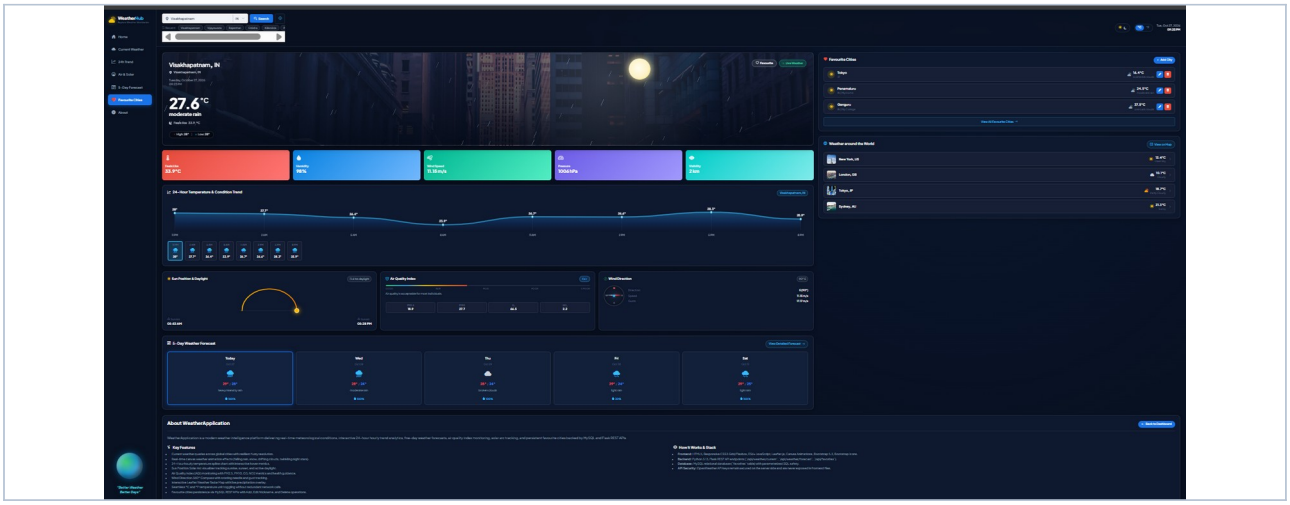


Figure A6: Complete Dashboard Layout

Full-page view of the WeatherHub dashboard.